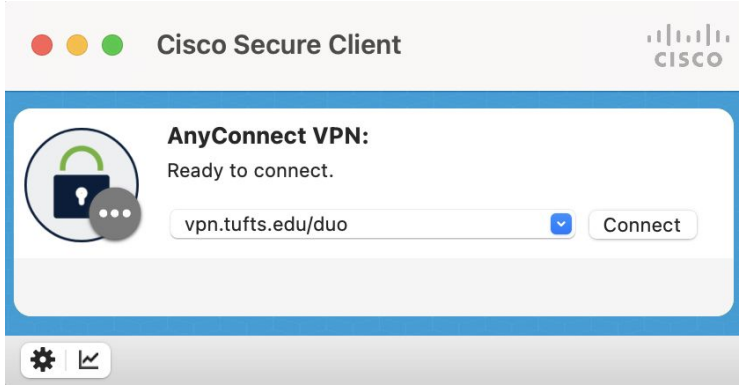
HPC

/cluster/tufts/c26f1data0221/

VPN





NEED if not on Tuft's Wifi

CLI and GUI

1. SSH

   ssh utln@login-prod.pax.tufts.edu

   <- Password

2. OnDemand

   https://ondemand-p01.pax.tufts.edu/pun/sys/dashboard



CLI Commands:

Local -> HPC Upload: (-r for folders)

```
Scp a_file.py utln@login-prod.pax.tufts.edu/cluster/tufts/c26f1data0221/shared
```

HPC -> HPC:

```
Cp a_fil.py /cluster/tufts/c26f1data0221/anywhere
```

Job Submission

Sbatch

- **squeue:** lists your jobs in the queue
- **sinfo**: lists the state of all computers in the

  HPC cluster

- **sbatch** submits batch jobs

`sbatch mygpujob.sh`

- Check Job

`mygpujob.sh`

```
#!/bin/bash -l
#SBATCH -J My_Job_Name   #job name
#SBATCH --time=00-00:20:00  #requested time (DD-HH:MM:SS)
#SBATCH -p gpu,preempt    #running on "batch" or "preempt" partition, wherever re
#SBATCH -N 1    #1 nodes #for many shared-memory programs,please leave -N as 1.
#SBATCH -n 2   #2 tasks total and 1 cpu per task, that gives you 2 cpu cores for
#SBATCH --mem=2g  #requesting 2GB of RAM total for the number of cpus you request
#SBATCH --gres=gpu:a100:1       #requesting 1 A100 GPU
#SBATCH --constraint="a100-80G" #only requesting the A100 GPU with 80GB memory
#SBATCH --output=MyJob.%j.%N.out  #saving standard output to file, %j=JOBID, %N=N
#SBATCH --error=MyJob.%j.%N.err   #saving standard error to file, %j=JOBID, %N=No
#SBATCH --mail-type=ALL    #email optitions
#SBATCH --mail-user=Your_Tufts_Email@tufts.edu #please make sure to use your own

# [commands_you_would_like_to_exe_on_the_compute_nodes]
# have a clean start. purge all loaded modules in current environment
module purge
# for example, running a python script
# load the module so the correct version python is available to you
module load miniforge/25.3.0
# when using GPUs, make sure to load the appropriate version of cuda toolkit to p
module load cuda/12.9.0
# If you have a conda env that you would like to use, activate it
source activate [target_env]
# run python script
python myscript.py #make sure myscript.py exists in the current directory
# make sure you save all plots, data, outputs generated to files in your script
# Don't forget to deactivate your conda env if you are using one
conda deactivate
```

Training Example

```
#!/bin/bash
# ============================================================
# train_something.sbatch or train_something.sh
# Submit with:
#   sbatch train_something.sbatch
# ============================================================

# --- Resource requests ------------------------------------
#SBATCH --job-name=im_training          # Name shown in squeue
#SBATCH --partition=gpu,preempt         # Queues with GPU access; preempt schedules faster
#SBATCH --requeue                      # Auto-requeue if preempted
#SBATCH --nodes=1                      # Single node
#SBATCH --ntasks=1                     # One process
#SBATCH --cpus-per-task=8              # ≥8 needed so mAP eval doesn't bottleneck (mp.Pool(8))
#SBATCH --gres=gpu:1                   # One GPU
#SBATCH --mem=32G                      # RAM
#SBATCH --time=04:00:00                # Wall-clock limit (HH:MM:SS)
#SBATCH -mail-user=my_email@gmail.com

# --- Output logs ------------------------------------------
#SBATCH --output=slurm_logs/%j_train.out   # stdout  (%j = job ID)
#SBATCH --error=slurm_logs/%j_train.err    # stderr

# ============================================================
# 0. Paths
# ============================================================
ROOT=/cluster/tufts/zhanglab_aida/hzhou05
REPO=$ROOT/dcase2026_task6_baseline
CONDA_ENV=/cluster/tufts/zhanglab_aida/conda/envs/clap
PYTHON=$CONDA_ENV/bin/python


# ============================================================
# 2. Load modules and activate conda env
# ============================================================
module purge
module load miniforge/25.3.0 cuda/12.9.0

source activate $CONDA_ENV

# ============================================================
# 3. Sanity checks — fail loud before wasting GPU time
# ============================================================
echo "--- Python: $($PYTHON --version)"
echo "--- GPU:"
nvidia-smi --query-gpu=name,driver_version --format=csv,noheader

# Confirm torch sees CUDA (not silently falling back to CPU)
$PYTHON -c "
import torch
assert torch.cuda.is_available(), 'CUDA not available — check driver/wheel mismatch'
print(f'torch {torch.__version__} | CUDA {torch.version.cuda} | device: {torch.cuda.get_device_name(0)}')
"

# ============================================================
# 4. Run — CWD must be $REPO (all config paths are relative)
# ============================================================
cd $REPO
mkdir -p slurm_logs   # in case it doesn't exist yet

$PYTHON src/train.py \
    -c config_castella.yml \
    -r results_pretraining/best_checkpoint.pth

echo "--- Job finished: $(date)"

conda deactivate
```